

MASTERING CLAUDE CODE

Complete Practical Guide — All Tips, Prompts & Workflows

AGENTIC AI

ALL PROMPTS

PRO TIPS

SDK & CLI

What We'll Cover

10 sections • Every prompt, workflow, and configuration option from the video

01

What is Claude Code?

Agentic AI, capabilities, installation

02

Initial Setup

Terminal, theme, GitHub app, dictation

03

Tip 1 — Codebase Q&A

Start here: explore before editing

04

Tip 2 — Editing Code

Plan→code, commit/push/PR, workflows

05

Tip 3 — Integrate Tools

Bash CLI tools + MCP servers

06

Tip 4 — Context (CLAUDE.md)

Memory, slash commands, hierarchy

07

Configuration Hierarchy

Project, global, enterprise policies

08

Key Bindings

All shortcuts and terminal tricks

09

SDK & Pipeline Mode

Headless, JSON, Unix pipes

10

Advanced: Parallel Sessions

tmux, SSH, Git worktrees

SECTION 01

What is Claude Code?

A new kind of agentic AI coding assistant

Claude Code is NOT Line Completion

Previous generation vs. the new generation

Previous Gen (Copilot-style)

✗ Autocomplete one line at a time

✗ Suggests a few lines based on cursor context

✗ You guide every edit manually

✗ IDE plugin that watches you type

Claude Code (Agentic)

✓ Builds entire features end-to-end

✓ Writes whole files and functions

✓ Fixes entire bugs autonomously

✓ Works across the whole repo

Key Capabilities

What makes Claude Code different



IDE-Agnostic

Works with VS Code, Xcode, JetBrains, Vim, Emacs — any IDE, any terminal



Code Stays Local

No remote index, no upload to any database, not trained on your code



Instant Start

No indexing wait. Open Claude Code and use it immediately — zero setup



General Purpose

Coding, debugging, Q&A, CI pipelines, incident response — one tool



Any Environment

Local machine, remote SSH, tmux, Docker — wherever your terminal runs



Fully Agentic

Plans, explores, edits, tests, and iterates — all without hand-holding

Installation

All you need is Node.js

```
npm install -g @anthropic-ai/claude-code
```

Requirement

Node.js installed on your machine

How to start

Type `claude` in any terminal

Works on

macOS, Linux, Windows (WSL)

IDE needed?

No — works with any IDE or none

Remote?

Yes — SSH, tmux, any remote environment

SECTION 02

Initial Setup

Run these commands once — right after install

Setup Step 1 — /terminal-setup

Make the terminal experience smoother

```
/terminal-setup
```

What it does

Enables Shift+Enter for new lines inside Claude Code

Without it

You need to type backslash + enter for multiline prompts

With it

Press Shift+Enter to start a new line — much more natural

Why it matters

Longer, more detailed prompts are easier to write and edit

Setup Step 2 — /theme

Choose your color theme

/theme

LIGHT

Standard light mode

Best for bright environments

DARK

Dark background, light text

Best for low-light / night coding

DALTONIZE

Color-blind friendly palette

Adjusted colors for accessibility

Setup Step 3 — /install-github-app

Connect Claude Code to GitHub

```
/install-github-app
```

What it installs

A GitHub App on your repos that Claude Code can interact with

@-mention on Issues

Tag Claude on any GitHub issue: "@claude investigate this bug" — Claude responds directly

@-mention on PRs

Tag Claude on any pull request — it will review, comment, or fix code

GitHub Actions

Pairs with CI pipelines — Claude can auto-label issues, summarize PRs, more

Setup Step 4 — Customize Allowed Tools

Stop being prompted for every command you use regularly

Problem: Claude asks permission for every bash command → constant approval interruptions → slow workflow

Solution: Add frequently-used commands to your allowed tools list → auto-approved, no interruption

EXAMPLES TO AUTO-APPROVE

```
Bash(git log)
Bash(npm test)
Bash(grep)
Bash(cat)
Bash(ls)
```

Where to configure:

- Project `.claude/settings.json` (shared)
- `~/.claude/settings.json` (personal global)
- Enterprise policy file (org-wide)
- Just tell Claude: 'remember to always allow this'

Setup Tip — macOS Dictation

Talk to Claude Code instead of typing

Boris: "I just hit the dictation key twice and speak my prompt — I talk to Claude like another engineer"

1 **Step 1** Open System Settings on macOS

2 **Step 2** Go to Accessibility → Dictation

3 **Step 3** Enable Dictation

4 **Step 4** Set a shortcut key (default: double-tap Fn or Mic key)

5 **Step 5** In Claude Code, double-tap your dictation key and speak your prompt

Works for detailed, specific prompts — speaking is faster than typing long context-rich instructions

SECTION 03

Tip 1 — Codebase Q&A

Start here. Explore before you edit.

Why Start with Codebase Q&A?

The #1 recommendation for new Claude Code users

Teaches you how to prompt

You learn the right level of detail and specificity without the risk of breaking code

Reveals Claude's capabilities

You discover what Claude can do in one shot vs. what needs more guidance

No risk — read only

Asking questions doesn't change any files. Safe way to explore the tool

Deeper than Cmd+F

Claude doesn't just text-search — it understands context, usage patterns, and history

Used for onboarding at Anthropic

New hires use Claude Code Q&A from day one — onboarding went from weeks to days

Works on any codebase instantly

No indexing, no setup. Open Claude Code and start asking — it explores live

Onboarding Impact at Anthropic

The real-world proof that Q&A-first works

BEFORE Claude Code

2–3 Weeks

- Ask engineers questions on the team
- Navigate an unfamiliar codebase alone
- Learn the toolchain and conventions
- Understand the architecture via docs

AFTER Claude Code

2–3 Days

- Ask Claude Code anything about the codebase
- Get deep answers with code examples instantly
- Understand Git history and decisions in seconds
- Architecture discovery in one session

80% of technical employees at Anthropic use Claude Code every day — including ML researchers using the notebook tool

How Codebase Q&A Works

No indexing, no remote database — live exploration

1

You ask a question

e.g. "How is UserAuth instantiated in this codebase?"

2

Claude searches files live

Uses its file search and bash tools to grep, read, and explore the repo in real time

3

Claude reasons over results

Synthesizes findings — not just a text search, but understanding relationships and patterns

4

Claude gives a deep answer

Provides code examples, file references, usage patterns — far deeper than a wiki or Cmd+F

Your code never leaves your machine. No remote index. No database. All exploration happens locally via Claude's tools.

Q&A Prompt — Class / Function Usage

Understand how any code is actually used

PROMPT

```
How is <ClassName> instantiated and used throughout the codebase?  
Show me concrete examples.
```

WHAT CLAUDE DOES

→ Searches for all files that import or instantiate the class

→ Finds actual usage patterns — not just text matches

→ Shows constructor arguments with real examples

→ Explains the role this class plays in the architecture

VARIATIONS

Q&A Prompt — Git History Archaeology

Understand WHY code is the way it is

PROMPT

```
Look through Git history — why does <functionName> have 15 arguments  
with these weird names? Who introduced them and what was the context?
```

WHAT CLAUDE DOES

- Runs git log on the specific file or function
- Finds the commits that introduced each argument
- Reads commit messages and linked issue numbers
- Fetches the GitHub issues via web fetch for full context
- Summarizes the history — who, when, why, what problem it solved

Q&A Prompt — Fetch GitHub Issue

Pull live context from GitHub into your session

PROMPT

```
Fetch GitHub issue #<number> and summarize the context and current status.
```

A

I

P

u

Q&A Prompt — Weekly Standup Prep

Boris uses this every Monday morning

PROMPT

```
What did I ship this week? Look through the Git log for my username.
```

Boris: "I do this every single Monday in our weekly standup. Claude looks through the log, knows my username, gives me a nice readout of everything I shipped. I copy-paste it into the doc."

What Claude does

Runs git log filtered by your username, looks at the last 7 days

Output

A formatted list of commits and features you shipped, ready to paste into Slack/docs

Extend it

Add: 'also summarize which PRs I reviewed' or 'what issues did I close?'

Real value

Saves 10+ minutes every standup — and never forget what you shipped

Codebase Q&A — All Use Cases

Everything you can ask before writing a single line of code

New engineer onboarding

Understand the repo structure, key abstractions, and conventions in hours not weeks

Legacy code archaeology

Understand why old code is written a certain way using Git history and issue context

Architecture discovery

Map out how services communicate, where data flows, and what each module does

Weekly standup prep

Auto-generate your 'what I shipped' summary from the Git log every Monday

Pre-task research

Before starting a feature, understand all related code and potential impact areas

Code review prep

Understand the history and rationale before reviewing someone else's PR

Debug investigation

Ask Claude to find all places a function is called and trace a bug path

Documentation gap filling

Ask questions your docs don't answer — Claude reads the actual code

SECTION 04

Tip 2 — Editing Code

From planning to committing — the full workflow

Claude Code's 3 Core Tools

A small set of tools that Claude strings together intelligently

Edit Files

Create, modify, or delete any file in your project. Claude writes targeted diffs — not full rewrites.

Examples:

- Create new component files
- Add a function to an existing module
- Fix a bug in a specific line
- Rename or restructure files

Run Bash

Execute any shell command. Claude uses this to run tests, check git state, install packages, and more.

Examples:

- Run npm test / pytest
- Check git log or git diff
- Install dependencies
- Run your CLI tools



Search Files

Find patterns across files using grep-style search. Claude uses this to explore before editing.

Examples:

- Find all usages of a function
- Locate config files
- Search for error strings
- Identify import patterns

You NEVER have to specify which tool to use. Just describe the goal — Claude strings them together automatically.

The Golden Rule — Always Plan First

"Ask it to think first" — the most important tip for code editing

Boris: "If you ask Claude to implement a 3,000-line feature, sometimes it gets it right — but sometimes the thing it builds is not at all what you wanted. The easiest fix: ask it to think first."

PLANNING PROMPT

```
Before you write any code, brainstorm ideas and make a plan.  
Run the plan by me and ask for my approval before you start writing code.
```

WHY THIS WORKS

- | | |
|--------------------------|--|
| Alignment | You see the plan before code is written — catch misunderstandings early |
| No wasted work | Avoid building the wrong thing and having to tear it down |
| No special mode | You don't need plan mode or any special tool — just ask Claude in plain language |
| Claude's instinct | Even without being told, Claude often reasons through a problem before acting |

The "commit, push, PR" Incantation

Three words that trigger a full Git workflow

PROMPT

`commit, push, PR`

WHAT CLAUDE DOES AUTOMATICALLY

- 1 Examines your git log** Looks at your repo's commit history to learn your commit message format and conventions
- 2 Writes the commit message** Crafts a commit message in your exact style — conventional commits, ticket numbers, etc.
- 3 Creates a branch if needed** Names the branch appropriately based on the work that was done
- 4 Pushes to remote** Pushes the branch to origin — no flags needed, it figures out the right remote
- 5 Opens a Pull Request on GitHub** Creates the PR with a description summarizing the changes — ready for review

Workflow 1 — Explore → Plan → Edit

The standard workflow for all non-trivial tasks

1 EXPLORE

Claude reads relevant files, searches for patterns, understands the current state

2 BRAINSTORM

Claude considers multiple approaches, trade-offs, edge cases

3 PLAN

→ "Before writing code, brainstorm ideas and make a plan."

Claude writes out a step-by-step implementation plan

4 APPROVAL

→ "Run the plan by me and ask for my approval."

Claude presents the plan and waits for your go-ahead — you can redirect here

5 EDIT

Claude implements the plan, making targeted file edits

6 VERIFY

Claude runs tests or checks its work before finishing

Workflow 2 — Iterate with Tests

Give Claude a way to check its work — it self-corrects automatically

```
Write unit tests for <feature>.  
Run them. If they fail, fix the code and run again.  
Iterate until all tests pass. Don't ask for approval between iterations.
```

HOW THE LOOP WORKS

1. Claude writes the implementation
2. Claude writes unit tests for the implementation
3. Claude runs the tests via bash
4. If tests fail: Claude reads the error output
5. Claude edits the code to fix the failure
6. Claude re-runs the tests
7. Repeats until all tests are green

Result: Claude reaches near-perfect test coverage and correct implementation in 2–3 iterations — without you approving each step

Workflow 3 — Iterate with Screenshots

Visual feedback loop for UI work — Puppeteer, iOS simulator

```
Build this web UI. [drag-drop mockup image into Claude]
Use the Puppeteer MCP server to screenshot the current state.
Compare to the mockup. Iterate until they match.
```

THE VISUAL ITERATION LOOP

1. You drag-drop a mockup image into Claude Code (multimodal input)
2. Claude builds an initial implementation
3. Claude uses Puppeteer (or iOS simulator) to take a screenshot
4. Claude visually compares the screenshot to your mockup
5. Claude identifies differences — layout, spacing, colors, typography
6. Claude edits the code to fix the differences
7. Claude re-screenshots and compares again
8. After 2–3 iterations: result is often near-perfect

Claude Code is Fully Multimodal

Images in the terminal — drag, drop, paste, or path

Boris: "Claude Code is fully multimodal — it's in the terminal so it's a little hard to discover, but you can drag and drop images, give a file path, or copy-paste an image in."

Drag & Drop

Drag an image file from Finder/Explorer directly into the terminal window

```
[drag image into terminal]
```

File Path

Give Claude the path to an image file on your filesystem

```
Here's the mockup: /path/to/mockup.png
```

Copy & Paste

Copy an image to clipboard and paste it into the Claude Code prompt

```
[Cmd+V / Ctrl+V in the prompt]
```

Primary use case: drag-drop a UI mockup, then use Puppeteer to screenshot and iterate until the implementation matches

Choosing the Right Workflow

Match the workflow to the task type

Task Type	Recommended Workflow	Key Prompt
Simple bug fix	Direct edit — no plan needed	Fix the bug in <file>:<line>
New feature (non-trivial)	Explore → Plan → Approval → Edit	Make a plan first, ask for approval
Unit test writing	Iterate with tests	Write tests. Run. Fix. Repeat until green.
Web UI from mockup	Iterate with screenshots (Puppeteer)	Screenshot via Puppeteer, compare, iterate
Mobile UI (iOS)	Iterate with simulator	Use iOS simulator to screenshot and iterate
CLI tool / script	Direct edit + bash validation	Write it, run it, fix errors, rerun
Large refactor	Plan + incremental edits	Plan first, one module at a time

SECTION 05

Tip 3 — Integrate Tools

Bash CLI tools and MCP servers — the force multiplier

Two Kinds of Tools

Claude Code supports both — just describe them and Claude figures out how to use them

Bash / CLI Tools

Any command-line tool your team uses. Tell Claude the name and it will run `--help` to learn the flags automatically.

Examples:

- Your internal test runner
- Deployment CLI
- Database migration tool
- Linter / formatter
- Any bash utility

MCP Tools

Model Context Protocol servers that extend Claude with external capabilities — databases, APIs, browsers, and more.

Examples:

- Puppeteer (browser automation)
- Database query tools
- Slack / Jira connectors
- Custom internal APIs
- File system watchers

Key principle: You never have to explain HOW to use a tool. Just tell Claude the tool exists — it will run `--help` or read docs itself.

Bash Tool — Prompt Pattern

How to introduce any CLI tool to Claude Code

TEMPLATE

```
Use the <cli-name> CLI to do <task>.  
Run `<cli-name> --help` to understand how to use it first.
```

REAL EXAMPLE

```
Use the barley CLI to run the integration tests for the payments module.  
Run `barley --help` to figure out the right flags.
```

WHAT HAPPENS

- Claude runs `barley --help` to learn the available commands and flags
- Claude identifies the right subcommand for integration tests
- Claude constructs the correct command and runs it
- Claude reads the output and reports results (or fixes failures)

MCP Tool — Prompt Pattern

How to use any MCP server in Claude Code

TEMPLATE

```
I've added the <server-name> MCP server.  
Use it to <task description>.
```

REAL EXAMPLE

```
I've added the Puppeteer MCP server.  
Screenshot the current state of the UI and compare it to the mockup I just dropped in.
```

MCP SETUP

Add via config

Add the server to .mcp.json in your project root

Or add at runtime

Tell Claude: 'I've added the X MCP server' and it will use it

Share with team

Check .mcp.json into source control — everyone gets it

Enterprise rollout

Add to enterprise config — all employees get it automatically

Sharing Tools via .mcp.json

Configure once — every engineer on the team benefits automatically

```
// .mcp.json (check this into source control)
{
  "mcpServers": {
    "puppeteer": {
      "command": "npx",
      "args": ["@anthropic/puppeteer-mcp"]
    },
    "postgres": {
      "command": "npx",
      "args": ["@anthropic/postgres-mcp", "--db-url", "postgresql://localhost/myapp"]
    }
  }
}
```

HOW IT WORKS

1. Check .mcp.json into your Git repository
2. Any engineer who clones the repo and runs Claude Code is prompted to install the MCP servers
3. One setup — zero per-engineer overhead
4. You can also put it in enterprise config for org-wide distribution

Real Example — Anthropic Apps Repo + Puppeteer

How Anthropic uses this pattern internally

Boris: "In our apps repo — the repo with all our web and apps code — there's a Puppeteer MCP server. We share it with the team via a .mcp.json checked in to the repo."

What's in the repo

.mcp.json with Puppeteer MCP server configuration checked in

Who gets it

Every engineer who opens Claude Code in that repo is prompted to install Puppeteer

What it enables

Automated end-to-end testing, screenshot iteration for UI work — zero per-engineer setup

The workflow

Engineer drops a UI mockup into Claude → Claude builds it → Screenshots via Puppeteer → Iterates until it matches

Without this pattern

Each engineer would have to manually install and configure Puppeteer themselves

The network effect

One engineer configures it → entire team benefits. This is the recommended way to share tools.

Tool Integration — Recommended Workflow

How to onboard your team onto a new codebase with tools

1

Inventory your team's tools

List every CLI tool and MCP server your team uses on this codebase

2

Add bash tools to CLAUDE.md

Document the command name, purpose, and common flags in the project CLAUDE.md

3

Add MCP servers to .mcp.json

Create or update .mcp.json in the project root with all relevant MCP server configs

4

Check both files into source control

Commit CLAUDE.md and .mcp.json so every engineer gets them on clone

5

Test with a fresh clone

Clone the repo fresh and open Claude Code — verify tools are available and prompts work

6

Document in onboarding

Add a note in your team onboarding guide pointing to CLAUDE.md and the available tools

SECTION 06

Tip 4 — Context (CLAUDE.md)

Give Claude more context → smarter decisions

What is CLAUDE.md?

The context file that makes Claude Code aware of your project

CLAUDE.md is a special file that Claude Code reads automatically at the start of EVERY session

HOW IT WORKS

- When you open Claude Code in a project directory, it looks for CLAUDE.md
- The contents of CLAUDE.md are included in the first user turn of every session
- Claude reads this before it does anything else — so it already knows your context
- No prompt needed — it's always there, always loaded
- Think of it as a briefing document that every Claude session reads before starting work

Boris: "As you give Claude more context, it'll do better. The more context, the smarter the decisions — because you have a ton of context in your head about your systems."

What to Put in CLAUDE.md

The right content makes a dramatic performance difference

Bash Commands

- Common test commands (npm test, pytest, etc.)
- Deployment commands
- Database migration commands
- Linting and formatting commands

MCP Tools

- Names of available MCP servers
- When to use each server
- Any special flags or configuration
- Authentication notes

Architecture

- Key architectural decisions
- Core modules and their responsibilities
- Data flow overview
- Important constraints and trade-offs

Important Files

- Main entry points
- Configuration files
- Core abstractions
- Files that must never be modified directly

Style Guide

- Naming conventions
- Code formatting preferences
- Patterns to follow
- Anti-patterns to avoid

Project Context

- What this project does
- Non-obvious background
- Key external dependencies
- Team conventions and norms

CLAUDE.md — Keep It Short

More isn't always better — context quality matters more than quantity

Boris: "Try to keep it pretty short, because if it gets too long, it's just going to use up a bunch of context, and it's usually not that useful."

✗ TOO LONG — Problems

- Wastes context window tokens
- Buries important info in noise
- Slows Claude down on every session
- Hard to maintain and keep accurate
- Like giving someone a 100-page manual instead of a cheat sheet

✓ JUST RIGHT — Principles

- Bullet points not paragraphs
- Only what's non-obvious
- Commands Claude will actually need
- Decisions that aren't in the code
- One section per topic — no repetition

CLAUDE.md — Location 1: Project Root

The primary, team-shared context file

```
<project-root>/CLAUDE.md
```

PROPERTIES

Loaded automatically

Yes — every single session in this project

Check into source control

YES — this is the team-shared file

Scope

All Claude Code sessions opened in this project directory

Recommended use

Common commands, style guide, architecture, MCP tools for the team

Who edits it

Any team member — treat it like team documentation

START HERE. Write this once, check it in, and every engineer on your team benefits immediately.

CLAUDE.md — Location 2: Local (Personal)

Your private preferences — not shared with team

```
<project-root>/.claude/CLAUDE.md  (or CLAUDE.local.md)
```

Loaded automatically

Yes — alongside the project root CLAUDE.md

Check into source control

NO — add to .gitignore

Scope

Only your sessions on this project

Recommended use

Personal workflow preferences, your own shortcuts, API keys, personal style overrides

Example contents

- My preferred test runner flags
- My local database URL
- Personal coding style notes
- Reminders about my personal setup

CLAUDE.md — Location 3: Nested Directories

Context loaded on-demand for specific subsystems

```
<project-root>/<subdir>/CLAUDE.md
```

Nested CLAUDE.md files are NOT loaded at startup — they are loaded automatically when Claude works in that specific directory

When loaded

When Claude reads, edits, or searches files in that subdirectory

Use case

Subsystem-specific context that would clutter the main CLAUDE.md

Examples

- /api/CLAUDE.md — API conventions and endpoints
- /frontend/CLAUDE.md — Component library and UI patterns
- /ml/CLAUDE.md — Model training commands and data pipeline docs

Benefit

Keeps the root CLAUDE.md lean while still providing deep context when needed

CLAUDE.md — Global & Enterprise Locations

Across all projects and across the entire org

Global (~/.claude/CLAUDE.md)

All your projects on this machine

Loaded:	Yes — on every Claude Code session you run, on any project
Use case:	Your personal preferences across all codebases — editor shortcuts, personal style • Always use TypeScript strict mode • Prefer functional patterns • My GitHub username is @sheker
Example:	
Check in:	No — personal only

Enterprise Root CLAUDE.md

All employees, all projects — managed by admin

Loaded:	Yes — auto-loaded for all users in the org
Use case:	Org-wide policies, coding standards, security requirements, approved tools • Company security policy: never log PII • Approved deployment process • Org-wide style guide
Example:	
Check in:	Yes — managed by admin in enterprise config path

CLAUDE.md — Full Hierarchy

All levels, loaded in order, merged together in context

Enterprise Root CLAUDE.md

Org-wide policies — cannot be overridden by employees

~/.claude/CLAUDE.md (Global)

Your personal preferences across all projects

<project-root>/CLAUDE.md (Project)

Team-shared context — check into source control ← START HERE

<project-root>/.claude/CLAUDE.md (Local)

Your personal project overrides — do NOT check in

<subdir>/CLAUDE.md (Nested)

Loaded on-demand when Claude works in that subdirectory

/memory — View All Memory Files

See exactly what context Claude has loaded in this session

/memory

WHAT IT SHOWS YOU

→ All CLAUDE.md files currently loaded in this session — in order (enterprise → global → project → local → nested)

→ The file path of each memory file so you know exactly what's being used

→ Whether each file is from enterprise policy, your home dir, or the project

→ Ability to click/select a specific memory file to view or edit it directly

EXAMPLE OUTPUT

Memory files loaded:

1. /etc/claude/enterprise.md [Enterprise Policy]
2. ~/.claude/CLAUDE.md [Global]
3. ./CLAUDE.md [Project — shared]
4. ~/.claude/CLAUDE.md [Local — personal]

— Save to Memory During a Session

Tell Claude to remember something — it writes it to CLAUDE.md automatically

```
# <thing you want Claude to remember>
```

HOW IT WORKS

1. Type # followed by what you want remembered
2. Claude asks which memory file to write it to (enterprise / global / project / local)
3. Claude adds it to the chosen CLAUDE.md file
4. It's available in every future session — permanent memory

EXAMPLE USES

```
# Always use snake_case for Python variables in this project
```

```
# The test command is: npm run test:integration
```

```
# Never modify files in the /generated directory
```

```
# The payments module uses event sourcing — don't use direct DB writes
```

Slash Commands — Custom Workflows

Save repeatable workflows as slash commands

LOCATION

```
.claude/commands/  
  label-github-issues.md  
  weekly-standup.md  
  deploy-staging.md  
  run-e2e-tests.md
```

USAGE

```
/label-github-issues  
/weekly-standup  
/deploy-staging
```

REAL EXAMPLES FROM ANTHROPIC

/label-github-issues

Runs automatically on new issues — Claude reads and labels them without human involvement. Triggered from a GitHub Action.

/weekly-standup

Generates a standup summary from your Git log — what you shipped, what's in review, what's blocked.

Enterprise slash commands

Shared via enterprise config — every employee gets the same set of /commands without any setup.

SECTION 07

Configuration Hierarchy

Projects, global, enterprise — a layered system for every scale

Configuration Hierarchy Overview

Four layers — each can be overridden by the layer below (except enterprise)

Enterprise Policy

Managed by admin — org-wide

Cannot be overridden by anyone

Global Config

~/.claude/config | ~/.claude/CLAUDE.md

Overrides defaults; cannot override enterprise

Project Config

.claude/config | CLAUDE.md (checked in)

Overrides global; cannot override enterprise

Local Project Config

.claude/config.local | .claude/CLAUDE.md (not checked in)

Most specific; cannot override enterprise

What Can Be Configured at Each Level

The same system applies to context, tools, commands, and permissions

Config Type	Project	Global	Enterprise
CLAUDE.md (context)	✓	✓	✓
Slash commands	✓	✓	✓
Allowed bash tools	✓	✓	✓
Blocked commands / URLs	✓	✓	✓ (non-overridable)
MCP servers (.mcp.json)	✓	—	✓
Auto-approved commands	✓	✓	✓
Enterprise policies	—	—	✓ only

Enterprise Configuration — Use Cases

What enterprise policy enables for large teams



Auto-approve team-wide commands

Add your standard test command so no engineer is ever prompted for approval
Example: auto-approve "npm run test:ci" for everyone



Block sensitive URLs permanently

Add internal URLs that should never be fetched — employees cannot override this
Example: block the production database admin panel URL



Share MCP servers org-wide

Add MCP server configs so every employee gets them on every project automatically
No individual setup required — instant org-wide rollout



Enforce coding standards

Write your org-wide style guide and security policies into the enterprise CLAUDE.md
Every Claude Code session reads it — consistent standards across all teams



Roll out slash commands

Share standard workflows (/deploy, /review, /standup) across all engineers via enterprise config
One update → everyone gets the new command



Security controls

Block access to sensitive bash commands that should never run in production
Example: block `rm -rf` or `DROP TABLE` patterns in certain environments

Project Configuration — Detail

What to configure at the project level and where to put it

```
.claude/  
  config           ← project settings (check in)  
  config.local     ← personal overrides (gitignore)  
  commands/       ← slash commands (check in)  
    standup.md  
    deploy.md  
CLAUDE.md         ← context/instructions (check in)  
.mcp.json         ← MCP servers (check in)
```

WHAT GOES WHERE

CLAUDE.md	Context, commands, style guide, architecture notes — team shared
config	Allowed/blocked tools, settings — team shared
config.local	Your personal tool preferences — do not check in
.mcp.json	MCP server configs — check in to share with team
commands/	Slash command markdown files — check in

The Network Effect of Shared Context

Start with shared project context — the highest ROI configuration

Boris: "Write context once and share it with everyone on the team — you get a network effect where one engineer does a little bit of work and everyone on the team benefits."

One Engineer

Spends 30 minutes writing CLAUDE.md and .mcp.json

Personal productivity improvement

Commits & Pushes

Checks the files into source control

Captured for the team permanently

Team Clones

Every engineer who pulls gets the config automatically

Team-wide productivity improvement

Team Improves It

Other engineers add their learnings, tools, and prompts

Config gets smarter over time

Compounding

Every new engineer gets the full accumulated knowledge from day 1

Onboarding keeps getting faster

SECTION 08

Key Bindings

Essential shortcuts — hard to discover in a terminal interface

Shift + Tab — Auto-Accept Edits Mode

Stop approving every file edit — let Claude work uninterrupted

Shift + Tab

What it does

Toggles auto-accept edits mode — file edits no longer require your manual approval

Bash still needs approval

Shell commands always still require approval — only file edits are auto-accepted

How to use

Press Shift+Tab once to enable — press again to disable

When to use

When Claude is clearly on the right track and you don't need to review every diff

Specific use case

During unit test iteration loops — Claude writes test, runs, fixes, repeats without interruption

Undo always works

If Claude makes a mistake, just ask it to undo — or use Escape and redirect

— Save to Memory (Keybinding)

The fastest way to teach Claude something new — right in the middle of a session

<thing to remember>

Boris: "Anytime you want Claude to remember something — for example if it's not using a tool correctly and you want it to use it correctly from then on — just type the # sign."

EXAMPLE MEMORIES

```
# Always run `npm run lint` before committing in this project
```

```
# Use the barley CLI for tests, not npm test
```

```
# The staging environment URL is https://staging.myapp.com
```

```
# Never edit files in /src/generated — they are auto-generated
```

```
# Prefer async/await over .then() chains in this codebase
```

! — Drop to Bash Mode

Run a command locally AND feed the output into Claude's context

! <bash command>

Runs locally

The command runs on your machine immediately — not through Claude

Goes into context

Both the command AND its output appear in Claude's context window on the next turn

Claude sees it

Claude can reference the output to make better decisions

Long-running commands

Use this for commands where you want to control timing but share results with Claude

EXAMPLES

```
! git status
! npm run build
! cat error.log | tail -50
```

@ — Mention Files and Folders

Pull specific files into context without asking Claude to find them

@ <file or folder path>

What it does

Adds the file or folder contents directly into the current conversation context

Files

@src/auth.ts — Claude reads that file's content immediately

Folders

@src/components — Claude gets a listing and can read individual files

When to use

When you know exactly which file is relevant and want Claude to focus on it

EXAMPLES

Speeds up work

No need for Claude to search — you direct its attention precisely

```
Fix the bug in @src/payments/processor.ts
Refactor @src/auth/ to use the new token pattern
How does @src/api/routes.ts relate to @src/middleware/?
```

Escape — Safe Stop Anytime

Stop Claude mid-action without corrupting your session

Escape

Always safe

Escape never corrupts the session. Press it at any point — mid-edit, mid-bash, mid-search

Stops current action

Whatever Claude is doing pauses immediately — cleanly

Use case: redirect

Claude suggested a 20-line edit but one line is wrong → Escape → tell it the correction → re-run

Use case: refine

Claude is going in the wrong direction → Escape → add more context → let it retry

Not permanent

After Escape, Claude is still in context — your conversation history is intact

Best practice

Use it freely — it's a power tool, not an emergency button

More Key Bindings — History, Output, Resume

Escape Escape

Jump back in history

Navigate to a previous message in the conversation — useful for retrying a different approach

Ctrl + R

Show full output

Expands the entire output — shows everything in Claude's context window, not just the summary

claude --resume

Resume previous session

Restart Claude Code and pick up exactly where you left off — full context preserved

```
claude --resume
```

claude --continue

Continue last session

Similar to --resume but continues the most recent session without a selection prompt

```
claude --continue
```

KEY BINDINGS — CHEAT SHEET

Everything you need to know — print and keep nearby

Shift + Tab

Toggle auto-accept edits mode

Bash still needs approval

<text>

Save to memory / CLAUDE.md

Choose which file to write to

! <command>

Run bash locally

Output enters Claude's context

@ <path>

Mention file or folder

Adds contents to context

Escape

Stop current action safely

Never corrupts session

Esc Esc

Jump back in history

Navigate previous messages

Ctrl + R

Show full output

View Claude's entire context

--resume

Resume previous session

claude --resume or --continue

SECTION 09

SDK & Pipeline Mode

claude -p: Claude as a super-intelligent Unix utility

What is the Claude Code SDK?

The `-p` flag turns Claude Code into a headless pipeline tool

Boris: "We use this in CI all the time, for incident response, in all sorts of pipelines. Think of it as a Unix utility. You give it a prompt, it gives you JSON. Pipe in, pipe out."

Non-interactive mode

`claude -p` runs Claude without opening the interactive terminal UI — pure stdin/stdout

Same underlying model

Exactly the same model and SDK as the interactive Claude Code — same power, no UI overhead

Unix-pipe compatible

Output to stdout, input from stdin, exit codes — integrates seamlessly with any shell pipeline

Structured output

JSON and streaming-JSON output formats — easy to parse, process, and integrate downstream

Tool-restricted mode

`--allowedTools` lets you restrict which tools Claude can use — safe for automated pipelines

Build-anything platform

CI/CD, incident response, automated reviews, data processing — the combinations are endless

SDK — Basic Usage & Flags

All the flags you need to know for headless Claude Code

```
claude -p "your prompt here" \  
  --allowedTools "Bash(git log),Bash(grep)" \  
  --output-format json
```

FLAGS REFERENCE

-p / --print	The prompt to send. Enables non-interactive (headless) mode.
--allowedTools	Comma-separated list of tools Claude can use. e.g. Bash(git log),Bash(grep)
--output-format json	Returns a single JSON object at the end. Easy to parse with jq.
--output-format stream-json	Streams JSON events in real time. Use for long-running tasks or live processing.
--no-permission-prompt	Skip all permission prompts. Useful in CI where there's no human present.
--max-turns	Limit the number of agent turns. Good for CI to prevent runaway loops.

SDK — Pipeline Examples

Real patterns for piping Claude into your workflows

Git Log → Sprint Summary

```
git log --oneline -20 | claude -p "Summarize what shipped this sprint" --output-format json | jq '.result'
```

Error Log → Root Cause

```
cat app.log | claude -p "What is anomalous or concerning in this log? What is the likely root cause?"
```

Sentry Events → Diagnosis

```
sentry-cli events list --project myapp | claude -p "Identify error patterns and suggest root causes"
```

Git Status → Commit Draft

```
git status | claude -p "Draft a commit message for these changes" --output-format json
```

Test Output → Failure Summary

```
npm test 2>&1 | claude -p "Summarize the test failures and suggest fixes for each one"
```

PR Diff → Code Review

```
git diff main...HEAD | claude -p "Review this diff. Flag bugs, style issues, and missing tests."
```

SDK — Use Cases at Anthropic

How Anthropic's own team uses headless Claude Code in production

GitHub Issue Auto-Labeling

A GitHub Action runs `/label-github-issues` slash command on every new issue. Claude reads the issue, understands the content, and applies the right labels — no human needed.

```
GitHub Actions + claude -p + /slash-command
```

CI Code Review

On every PR, Claude reviews the diff for bugs, style violations, and missing test coverage. Results posted as a GitHub comment.

```
GitHub Actions + git diff | claude -p
```

Incident Response

During on-call incidents, engineers pipe log dumps and error traces to Claude to get an immediate diagnosis and suggested remediation steps.

```
PagerDuty webhook + log files | claude -p
```

Weekly Standup Summaries

Each engineer runs a personal script every Monday — `git log` piped to Claude returns a standup-ready list of shipped work.

```
cron job + git log | claude -p
```

SECTION 10

Advanced — Parallel Sessions

Run multiple Claude instances simultaneously for 10× throughput

The Power User Pattern

How advanced Claude Code users work at maximum throughput

Boris: "I'm sort of a Claude normie — I'll have one Claude running at a time. When I look at power users in and out of Anthropic, almost always they have SSH sessions, tmux tunnels, multiple checkouts, Claudes running in parallel."

tmux with multiple panes

Split terminal into panes — each pane has its own Claude session. Work on 3-5 tasks simultaneously from one screen.

SSH remote sessions

Claude runs on a powerful remote server via SSH. Your laptop just sends prompts. Great for heavy computation.

Multiple repo checkouts

Clone the same repo 3-4 times into separate directories. Each Claude has its own isolated working copy.

Git worktrees (recommended)

Git's built-in mechanism for multiple working trees from one repo. Clean isolation without full clones.

Git Worktrees — Isolated Parallel Claude Sessions

The cleanest way to run multiple Claude instances on one repo

WHY WORKTREES?

Clean isolation

Each Claude gets its own working directory — edits don't collide

Same Git history

All worktrees share the same .git directory — one repo, multiple views

No duplicate data

Far lighter than cloning — just a new working tree, not a full copy

Independent branches

Each worktree can be on a different branch — run multiple features simultaneously

SETUP COMMANDS

```
# Create isolated worktrees for each parallel task
```

```
git worktree add ../feature-auth origin/main
git worktree add ../feature-billing origin/main
git worktree add ../bugfix-api origin/main
```

```
# Open Claude in each (use tmux split panes)
```

```
cd ../feature-auth && claude # pane 1
cd ../feature-billing && claude # pane 2
cd ../bugfix-api && claude # pane 3
```

```
# When done, remove the worktree
```

```
git worktree remove ../feature-auth
```

tmux Setup for Parallel Claude

The terminal multiplexer that power users rely on

```
# Start a new tmux session
```

```
tmux new-session -s claude-work
```

```
# Split into panes (Ctrl+B then % for vertical, " for horizontal)
```

```
# In each pane, navigate to a different worktree and start Claude
```

```
# Pane 1: Feature A
```

```
cd ~/projects/myapp-feature-auth && claude
```

```
# Pane 2: Feature B (Ctrl+B then arrow to switch panes)
```

```
cd ~/projects/myapp-feature-billing && claude
```

```
# Pane 3: Bug fix
```

```
cd ~/projects/myapp-bugfix && claude
```

```
# Reattach to session later
```

```
tmux attach-session -t claude-work
```

KEY TMUX BINDINGS

Ctrl+B	%	Split pane vertically (side by side)
Ctrl+B	"	Split pane horizontally (top/bottom)
Ctrl+B	arrow	Switch between panes
Ctrl+B	z	Zoom current pane to full screen
Ctrl+B	d	Detach session (leaves Claude running!)

SECTION 11

All Prompts — Reference

Every prompt pattern demonstrated in the video

All Prompts — Codebase Q&A (Part 1)

Copy-paste ready

CATEGORY: Q&A — CODEBASE EXPLORATION

Q&A

How is <ClassName> instantiated and used throughout the codebase?
Show me concrete examples.

Q&A

Look through Git history – why does <functionName> have 15 arguments with these weird names?
Who introduced them and what was the context?

Q&A

Fetch GitHub issue #<number> and summarize the context, discussion, and current status.

Q&A

What did I ship this week? Look through the Git log for my username: <your-username>.

All Prompts — Codebase Q&A (Part 2)

More exploration and research prompts

Q&A

Explain the overall architecture of this codebase. What are the main modules and how do they interact?

Q&A

Where is <config-value> set and what code paths read it? Trace it through the system.

Q&A

What tests exist for <module>? Are there gaps in coverage? What would you add?

Q&A

Find all places in the code that call <external-API>. Are any of them missing error handling?

All Prompts — Editing & Planning

The core code-editing prompt patterns

PLAN

Before you write any code, brainstorm ideas and make a plan.
Run the plan by me and ask for my approval before you start writing code.

COMMIT

commit, push, PR

FEAT

Implement <feature description>.
Before coding: explore the relevant code, make a plan, get my approval.

FIX

Fix the bug described in @<file>:<line>.
Before editing: explain what you think is causing it.

REFAC

Refactor <module> to use <new-pattern>.
Don't change behavior — write tests first so we can verify.

All Prompts — Iteration Workflows

Tests and screenshots — the self-correction loops

TESTS

```
Write unit tests for <feature>.
Run them. If they fail, fix the code and run again.
Don't ask for approval between iterations – iterate until all tests pass.
```

INTEG

```
Write integration tests covering <scenario>.
Use the test runner: <command>. Iterate until all pass.
```

UI

```
Build this web UI. [drag-drop mockup image]
Use the Puppeteer MCP server to screenshot the result.
Compare to the mockup. Iterate until they match visually.
```

IOS

```
Build this mobile UI. [drag-drop mockup]
Screenshot via the iOS simulator. Iterate until it matches the design.
```

All Prompts — Tools & Memory

Introducing tools and saving context

BASH

```
Use the <cli-name> CLI to do <task>.  
Run `<cli-name> --help` to understand the flags first.
```

MCP

```
I've added the <server-name> MCP server.  
Use it to <task description>.
```

MEM

```
# Always use <pattern/convention> in this project
```

MEM

```
# The test command for this repo is: <command>
```

MEM

```
# Never edit files in <directory> – they are auto-generated
```

/CMD

```
/memory – view all loaded context files for this session
```

All Prompts — SDK & Pipeline

Headless Claude Code in shell pipelines

SDK

```
claude -p "your prompt" --allowedTools "Bash(git log)" --output-format json
```

PIPE

```
git log --oneline -20 | claude -p "Summarize what shipped" --output-format json | jq '.result'
```

PIPE

```
cat app.log | claude -p "What is anomalous or concerning in this log?"
```

PIPE

```
git diff main...HEAD | claude -p "Review this diff. Flag bugs and missing tests."
```

PIPE

```
npm test 2>&1 | claude -p "Summarize failures and suggest fixes for each one"
```

CI

```
sentry-cli events list | claude -p "Identify error patterns and suggest root causes"
```

SECTION 12

Summary

Key principles and getting started path

10 Principles for Mastering Claude Code

Internalize these and you'll get 10× more out of every session

01

Start with Q&A before editing — learn Claude's capabilities first

02

Always ask Claude to plan before writing code — catch misalignment early

03

Give Claude a feedback tool (tests, screenshots) — it will self-correct

04

'commit, push, PR' is all you need to say — Claude handles the rest

05

CLAUDE.md is your force multiplier — write it once, share with the team

06

Keep CLAUDE.md short — quality over quantity in context

07

Use # to save learnings mid-session — they persist forever

08

Escape is always safe — use it freely to redirect mid-action

09

claude -p turns Claude into a Unix utility — build pipelines

10

Run parallel sessions with Git worktrees + tmux to multiply throughput

Getting Started — Beginner to Advanced

Your roadmap for the first week with Claude Code

Day 1

Install & Ask Questions

`npm install -g @anthropic-ai/claude-code` • Run `/terminal-setup` and `/theme` • Open your project and ask: 'How does X work?' • Spend the first session only asking questions — no editing

Day 2–3

First Edits + Git Workflow

Ask Claude to fix a small bug — watch how it works • Try: 'Make a plan first, then get my approval' • Use 'commit, push, PR' after your first edit • Start a `CLAUDE.md` in your project root

Week 1

Context + Tools

Complete your `CLAUDE.md` (commands, style, architecture) • Add your team's CLI tools and MCP servers • Learn the key bindings: `Shift+Tab`, `#`, `!`, `Escape` • Try `iterate-with-tests` on a feature

Advanced

SDK + Parallel + Enterprise

Use `claude -p` in a pipeline or CI workflow • Set up Git worktrees for parallel Claude sessions • Configure enterprise or team-wide settings • Build a custom slash command for your team's workflow

Q&A Highlights — From Boris's Talk

Key questions and answers from the session

Q: Why a CLI tool instead of an IDE plugin?

A: Two reasons: (1) Engineers at Anthropic use many different IDEs — terminal is the common denominator. (2) By end of year, people may not use IDEs at all given model progress. Terminal-first is the future.

Q: Is Claude Code multimodal?

A: Yes — fully multimodal from day one. Drag-drop images in, give a file path, or copy-paste. Use for UI mockups, Puppeteer screenshots, design references — whatever you need.

Q: How does Claude Code keep bash commands safe?

A: Static analysis determines read-only vs. state-changing commands. A complex tiered permission system allows allow-listing and block-listing at multiple levels. Enterprise admins have full control.

Q: What was the hardest part to build?

A: Making bash commands safe without being annoying. Bash can change system state in unexpected ways — balancing safety and productivity across diverse codebases (not everyone runs Docker) was very tricky.

Q: Do you use Claude Code for ML / research?

A: Yes — both engineers and researchers at Anthropic use it daily. Researchers use it with the notebook tool to edit and run Jupyter notebooks. ~80% of technical Anthropic staff use it every day.

QUICK REFERENCE CARD

Mastering Claude Code — All Key Actions

START

Q&A before editing — never jump straight to code

PLAN

"Make a plan, get approval, then write code"

COMMIT

"commit, push, PR" — Claude handles everything

MEMORY

"# <text>" → permanently saved to CLAUDE.md

STOP

Escape — always safe, no session corruption

ITERATE

Give Claude tests or screenshots → near-perfect

CONTEXT

CLAUDE.md — check in, share with team

TOOLS

.mcp.json → shared MCP servers for the team

PIPELINE

claude -p "prompt" --output-format json

PARALLEL

Git worktrees + tmux = 10× throughput

BINDINGS

Shift+Tab, !, @, Esc, Ctrl+R, --resume

CONFIG

Enterprise → Global → Project → Local